# Nat's Personal AI OS — Build Guide

Sep 30, 2026 · @nat

## What you're building

You're building his system with his look, but aimed at your life: school, SMC, CS, football and gym. Your own spec (my-spec.md) is the brief. His course gives the structure, security rules and tests.

The system works in four layers, the same as his:

1. **Capture**: you type (later, speak) anything into one box.
2. **AI**: Claude works out what it is (task, event, weekly target, resource, idea), the subject, the deadline and how urgent it is.
3. **Memory**: everything is saved in your own Supabase database.
4. **Dashboard**: shows Today, Upcoming, This Week, Calendar and Ask AI.

The one rule for Claude Code: **where my-spec.md and the course disagree, my-spec.md wins.**

Realistic time: one evening for setup, then 2–4 sessions of building and fixing. He took two weeks; you're skipping finance, nutrition and business features, so it's much less.

## Nothing here is final

This plan will change as you use it, and that's expected. Steps 11–12 (Calendar, Whoop, study pipelines) are starting points, not fixed designs. Half-formed ideas live in `assets/ideas.md` until you're ready for them.

To change or add anything, at any point:

1. Add or edit the idea in `assets/ideas.md` (or change `my-spec.md` if it changes what you want overall).
2. Tell Claude Code: *"Turn the [idea] from ideas.md into a stage. Propose a plan first, don't build yet."*
3. Read its plan, change what you don't like, then say "go".
4. Test it, the same way as every checkpoint.

To change something already built, just describe it: *"Make the Today card bigger"* or *"Weekly football target should be 3, not 4."* Do one change at a time so you always know what broke.

## Your schedule: about 4 sittings

You can do several steps in one sitting. The rule is: finish a step, tick its checkpoint, type `/clear` in Claude Code, then start the next.

| Sitting | Steps | Rough time |
|---|---|---|
| 1 | Steps 1–5: setup + core dashboard | 2–3 h |
| 2 | Design pass + Step 6 AI capture + Step 7 Ask AI | 2–3 h |
| 3 | Step 8 put it online + Step 9 Telegram | 2 h |
| 4 | Step 10 guides + Step 11 Google Calendar and Whoop | 2–3 h |
| Later | Step 12 study pipelines + anything from ideas.md | when ready |

If you hit your Max usage limit mid-sitting, it resets every 5 hours. Take a break and carry on.

## His 7 days vs your steps

You don't do his lessons one by one. Every lesson is covered by a step below, done by Claude Code, or skipped on purpose.

| His course | Your step | Who does it |
|---|---|---|
| Start Here (accounts, kit) | Steps 1–2 | You |
| Day 1: define your OS | Already done: my-spec.md | Done |
| Day 2: Supabase memory | Step 4 | You run SQL; Claude Code adds tables |
| Day 3: dashboard | Steps 3 + 5 | Claude Code (you give screenshots) |
| Day 4: capture + AI | Step 6 | Claude Code |
| Day 4: Telegram + voice | Step 9 | Claude Code + you set up the bot |
| Day 5: daily cockpit | Step 5 (Today, This Week, routines) | Claude Code |

| His course | Your step | Who does it |
|---|---|---|
| Day 6: journal + brain | Step 7 (Ask AI) + Step 10 (guides); journal later | Claude Code |
| Day 7: secure, back up, ship | Step 8 | Claude Code + you check |
| Your Notion guides | Step 10 | You export; Claude Code imports |
| Finance, nutrition, CRM | — | Skip |

Every "Checkpoint" lesson in his course becomes a short checklist at the end of each step. You tick it off; if something fails, tell Claude Code.

## His asset kit: where each file is used

All 7 files go in `assets/` and every one is used; the prompts in Steps 5–8 tell Claude Code to read them. You only need to open two yourself (the schema and the tests).

| File | What it is | Used in | Your version |
|---|---|---|---|
| `supabase-schema.sql` | Database tables + security rules | Step 4: you run it | Use as-is. Claude Code adds your extra tables in Step 5 |
| `architecture-prompt.txt` | His master build instructions | Step 5 | Use as-is; my-spec.md overrides it (journal and brain wait) |
| `task-classification-prompt.txt` | How the AI sorts captured text | Step 6 | Claude Code extends it: events, weekly targets, resources, ideas, not just tasks |
| `daily-review-prompts.txt` | Planning and review prompts | Step 7 | Its "AI PRIORITY" and "AI STRATEGY" prompts become Ask AI's rules |
| `acceptance-tests.csv` | 24 tests (T01–T24) | Checkpoints in Steps 5–8 | See the test map below; skip journal and privacy-card tests |

| File | What it is | Used in | Your version |
|---|---|---|---|
| `security-backup-checklist.txt` | Security + backup checks | Step 8 | Use as-is; ignore Telegram lines until you add it |
| `v1-requirements.csv` | Day 1 planning worksheet | Reference only | Already replaced by my-spec.md |

**Which tests belong to which step:**

| Step | Tests from acceptance-tests.csv |
|---|---|
| Step 5 (core) | T01, T05, T09, T10, T11, T12, T13, T14, T15, T16, T19, T22, T23 |
| Step 6 (AI capture) | T03, T04, T06, T07, T08 |
| Step 8 (secure + ship) | T02, T21, T24 |
| Later (journal) | T17, T18 |
| Skip | T20 (hidden finance card; you have none) |

At each checkpoint you can just say to Claude Code: *"Run tests T03–T08 from assets/acceptance-tests.csv, tell me which pass, and fix any that fail."*

## Step 1: Get accounts and tools ready (30 min)

- [ ] **Claude Code**: needs a Claude Pro or Max plan. Install it, then check it runs by typing `claude` in a terminal.
- [ ] **Node.js**: install the LTS version from nodejs.org (Claude Code and the app both need it).
- [ ] **VS Code**: your editor. Open the terminal inside it with Ctrl+ `(or Cmd+` on Mac).
- [ ] **Supabase**: free account at supabase.com. Create a project called `personal-os`, pick the London region, and save the database password somewhere safe.
- [ ] **Anthropic API key**: at console.anthropic.com. This powers the AI capture and Ask AI. It's pay-as-you-go, so **set a monthly spend limit** straight away. A parent may need to set up billing.
- [ ] **GitHub** (optional now, needed in Step 8): free account for backing up your code and putting it online.

Don't paste any keys into chats, screenshots or code files. They only ever go in `.env.local` (Step 2).

## Step 2: Set up the project folder (15 min)

Make a folder called `personal-os` and put everything Claude Code needs to read inside `assets/`:

```
personal-os/
  assets/
    my-spec.md                      ← your "My Personal AI Dashboard" doc
    v1-requirements.csv
    architecture-prompt.txt
    supabase-schema.sql
    task-classification-prompt.txt
    acceptance-tests.csv
    security-backup-checklist.txt
    daily-review-prompts.txt
    screenshots/                    ← Step 3
    notion-guides/                  ← Step 10 (your Notion export)
```

- [ ] Copy each file from his Free Asset Kit page into a text file with the exact name above.
- [ ] Save your spec as `my-spec.md`.
- [ ] Open the `personal-os` folder in VS Code.

Claude Code will create `.env.local` (where your keys go) and add it to `.gitignore` in Step 5. You'll paste these values into it:

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
ANTHROPIC_API_KEY=
```

## Step 3: Capture his look (20 min)

He designed the dashboard in Claude Design first, then built it in Claude Code. You'll do the same using screenshots of his video.

- [ ] Pause the video at each of these and screenshot the full dashboard (hide YouTube controls first):

| Time | What to capture | Save as |
|---|---|---|
| 7:11 | Home dashboard, whole screen | `01-home.png` |

| Time | What to capture | Save as |
|---|---|---|
| 7:40 | Key tasks card close-up | `02-key-tasks.png` |
| 8:20 | Daily habits card | `03-habits.png` |
| 13:23 | Calendar + goals | `04-calendar-goals.png` |
| 14:38 | Kanban board | `05-kanban.png` |
| 14:49 | Brain / categories view | `06-brain.png` |

- [ ] Save them all in `assets/screenshots/`.
- [ ] **Optional but recommended**: drop the screenshots into Claude Design with: *"Redesign this dashboard in the same style for a sixth-form student. Sections: Today, Upcoming, This Week (target progress like Football 2/4), Calendar, Quick Capture, Ask AI. Categories: Study, Athletic, Personal."* Iterate until you like it, then export it to Claude Code (or screenshot your final mock into `assets/screenshots/`).

The screenshots are for style only: colours, fonts, card shapes, spacing. The content comes from your spec.

## Step 4: Set up the database (15 min)

This is his Day 2. The base tables come from his schema; Claude Code adds your extra ones in Step 5.

- [ ] In Supabase, open **SQL Editor → New query**, paste all of `supabase-schema.sql`, and click **Run**. It should say "Success. No rows returned."
- [ ] Check it worked: run `select table_name from information_schema.tables where table_schema = 'public';` You should see 8 tables, including `tasks`, `habits`, `goals` and `memory_items`.
- [ ] **Authentication → Users → Add user**: create yourself with your email and a password. This is your login.
- [ ] **Project Settings → API**: copy the Project URL and the `anon` public key. Keep them ready for `.env.local`.

Never copy the `service_role` key into the app. It bypasses all your security.

Run the schema **once only**. If you run it again, it stops with "policy … already exists". That just means it's already installed, so nothing is broken.

# Step 5: Build the core dashboard (1–2 sessions)

This covers his Days 3 and 5. In the VS Code terminal, run `claude` inside `personal-os`, then paste:

```
Read every file in /assets, including /assets/screenshots.
assets/my-spec.md is my real brief. Where it conflicts with the
course files, my-spec.md wins. Use the course files for structure,
security rules (security-backup-checklist.txt) and tests
(acceptance-tests.csv).

We're building in stages. This is STAGE 1 only. Stop when done.

STACK: Next.js + Supabase JS client. Create .env.local with
NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY and
ANTHROPIC_API_KEY (blank, I'll fill them) and add it to .gitignore.
The schema in assets/supabase-schema.sql is already installed.

SCHEMA: first propose additions (show me the SQL, wait for my OK) for:
- flexible deadlines: a window like "this weekend" or "before Wed"
- must-do vs nice-to-do on tasks
- weekly targets with progress (e.g. Football 4/week, SMC 3/week)
- routines that show only on their day and never go overdue
- fixed recurring events (school Mon–Fri 08:45–16:00,
  CS tutor Wednesdays)
- resources (links, videos, GoodNotes links) separate from tasks
- standing context facts about me for the AI to read
Keep row level security on every new table.

CATEGORIES: Study, Athletic, Personal only. Subjects (Maths,
Further Maths, Physics, CS, SMC, ACT, Football, Gym) are tags
inside those, not top-level categories.

HOME (match the layout, colours, fonts and card style in the
screenshots as closely as possible):
- Greeting with my name + date
- TODAY: 3–6 items, must-do first, then nice-to-do
- UPCOMING: deadlines and events for the next 14 days
- THIS WEEK: weekly target progress bars (Football 2/4 etc.)
- CALENDAR: fixed events this week
- ROUTINES: morning/night checklists, today only
- QUICK CAPTURE box (saves plain text to inbox for now)
- ASK AI box (placeholder for now)

OTHER PAGES: Tasks (list + kanban: inbox / today / this week /
later), Subjects (one page per subject: its tasks + resources),
```

```
Inbox (no-date ideas), Archive (completed).
Completing a task archives it, never deletes it.
Mobile friendly. Clear loading, empty and error states.
Skip the journal module in architecture-prompt.txt for now.
The Subjects page replaces his Brain view.

SEED for my user: school event, CS tutor event, weekly targets
(Football 4, SMC 3, Gym 3), Morning + Night routines, and
5 realistic sample tasks.

When finished: run the app, check yourself against the relevant
rows of acceptance-tests.csv, list anything failing, and give me
the exact commands to run it.
```

While it works:

- [ ] Read and approve the schema SQL it proposes. If it asks you to run SQL yourself, paste it into Supabase's SQL Editor.
- [ ] Paste your keys into `.env.local` when asked.
- [ ] Run `npm run dev` and open `localhost:3000`. Log in with the user from Step 4.

**Checkpoint (his Day 3 + Day 5 walkthrough):**

- [ ] Can I see today's items within 5 seconds of opening it?
- [ ] Add a task, refresh: is it still there, exactly once?
- [ ] Move a task on the kanban, refresh, check the list view: moved, not duplicated?
- [ ] Complete a task: gone from Today, visible in Archive?
- [ ] Tick a routine item: does it reset tomorrow without an "overdue" pile?
- [ ] Log a football session: does This Week go from 0/4 to 1/4?
- [ ] Log out: can I still see any data? (Should be no.)
- [ ] Does it look OK at phone width? (Drag the browser narrow.)

## Step 6: Add AI capture (1 session)

This is his Day 4 and the heart of your spec: type anything, the AI sorts it. Only start once every Step 5 checkbox passes.

```
STAGE 2: AI capture.

Quick Capture sends text to a server route that calls the Anthropic
API (ANTHROPIC_API_KEY from .env.local — server only, never in
browser code). Pass today's date and my timezone (Europe/London).
```

```
Adapt assets/task-classification-prompt.txt so the AI decides whether
the input is:
- a task (with subject, must-do or nice-to-do, deadline or window,
  rough duration)
- progress on a weekly target ("did football today" → Football +1)
- a fixed event ("CS tutor moved to Thursday 5pm")
- a resource to save (link, video, book, GoodNotes link)
- a no-date inbox idea ("look up that philosophy book")
- a context fact about me ("SMC is 7 October")

It must only use what's in the input plus my context facts — never
invent deadlines. Show me the proposal as an editable card; nothing is
saved until I confirm. Keep my original words in source_text.
If the AI call fails, save the raw text to Inbox with a visible
"not sorted yet" label. Warn me about likely duplicates.

Test it with the capture rows in acceptance-tests.csv and these:
"Physics homework due Friday"
"Need to watch the CS hash-table video before Wednesday, 1h20"
"Did 30 mins of juggling and first touch"
"I still don't understand vectors properly"
"SMC session sometime this weekend"
```

**Checkpoint (his capture reliability test):**

- [ ] A clear task with a date → sensible proposal, right subject.
- [ ] A vague idea → lands in Inbox, not forced onto a date.
- [ ] "Did football today" → This Week goes up by one; no new task created.
- [ ] Same text twice → duplicate warning.
- [ ] Blank out the API key temporarily → you see an error, and the text is still saved to Inbox.

## Step 7: Add the Ask AI planner (1 session)

This replaces his Day 6 "brain" with what your spec asks for: a planner that answers questions from your own data.

```
STAGE 3: Ask AI.

Make the Ask AI box on Home a real assistant. On each question, send
the Anthropic API: my open tasks, this week's target progress,
upcoming events (14 days), routines, inbox items, context facts,
today's date/time, and what I completed recently.

Base its system prompt on the "AI PRIORITY PROMPT" and "AI STRATEGY
```

```
PROMPT" in assets/daily-review-prompts.txt, plus:
- Give a short, realistic plan, not a long list. Max ~4 items with
  minutes each, e.g. "1. Physics homework — 35 min".
- School deadlines come before training; training before nice-to-dos.
- Account for school hours and the CS tutor on Wednesdays.
- Say what it's unsure about or missing.
- Recommend only. Never create, edit or complete records without
  my confirmation.
- If I tell it a fact about me ("remember my match is Saturday
  at 10"), offer to save it as a context fact; save only after I confirm.

Add one-tap buttons for: "What should I do tonight?",
"Plan my week", "What have I been postponing?",
"What's coming up that I might forget?"
```

**Checkpoint:**

- [ ] "What should I do tonight?" gives 3–4 items with times that fit a school evening.
- [ ] Every item it suggests really exists in your data. Nothing made up.
- [ ] "How am I doing with football this week?" matches the This Week bar.
- [ ] Asking it something doesn't change any of your tasks.

## Step 8: Secure, back up and put it online (1 session)

This is his Day 7. Putting it online is what lets you use it on your phone and iPad, not just your laptop.

```
STAGE 4: Security, backup, deploy.

1. Go through every item in assets/security-backup-checklist.txt
   and tell me which pass and which fail. Fix the failures.
2. Search the code for any secret keys that could reach the browser.
3. Add an "Export my data" button that downloads all my tables
   as JSON, so I can back up in one click.
4. Help me put the code on GitHub (private repo, .env.local excluded)
   and deploy it to Vercel, with my keys set in Vercel's environment
   settings. Walk me through each click.
5. Make it installable on my phone/iPad home screen (PWA).
```

**Checkpoint:**

- [ ] Security checklist: every item passes.
- [ ] T02: make a second test user in Supabase, log in as them, and check they see none of your data.

- [ ] T21: Claude Code confirms no secret keys appear in the browser code.
- [ ] The live Vercel link works, and you have to log in to see anything.
- [ ] Added to your phone and iPad home screens.
- [ ] First backup exported and saved somewhere safe (e.g. Google Drive). Repeat weekly.
- [ ] T24: ask Claude Code to restore that backup into a spare Supabase project and check the task counts match.

## Step 9: Add your Telegram assistant (1 session)

This is his headline feature: your phone's Action button opens a chat with your assistant. It's the same brain and database as the dashboard, just a second door. It needs the app online, so it comes after Step 8.

```
STAGE: Telegram assistant.

Create a Telegram bot (walk me through BotFather). Webhook on my
Vercel app, verified with a secret. The bot must ONLY respond to my
Telegram user ID — ignore everyone else.

Every message goes through the SAME logic as the dashboard:
- New info (task, event, target progress, fact about me) → reply
  with the proposal and ✅ Save / ✏️ Edit buttons.
- A question → answer it using Ask AI's logic and my data.
- Voice notes: transcribe server-side (OpenAI Whisper, key in
  Vercel env), then handle as text.

Scheduled messages (Vercel cron, Europe/London time):
- 07:00 morning brief: today's must-dos, events, targets left.
- Reminders before deadlines, and "leave in X min" before events
  that have a travel time saved.

Reuse the existing capture and Ask AI code — no duplicate logic.
Run the Telegram lines in assets/security-backup-checklist.txt.
```

**Action button (2 min):** in the Shortcuts app, make a shortcut with the action **Open URLs** and the link `tg://resolve?domain=YOURBOTNAME`. Then go to Settings → Action Button → Shortcut and pick it.

To skip Whisper and the extra API key, just use the microphone on your iPhone keyboard to dictate into Telegram.

**Checkpoint:**

- [ ] Action button → bot chat opens in under 2 seconds.

- [ ] "Physics homework due Friday" → proposal → ✅ → it appears on the dashboard.
- [ ] "What should I do tonight?" → same answer quality as Ask AI.
- [ ] A message from a friend's Telegram account gets ignored.
- [ ] Morning brief arrives at 07:00.

## Step 10: Bring in your Notion guides (1 session)

Your Notion Life Mastery Dashboard is a library of how-to guides. It becomes a Guides section: the dashboard says *what* to do today, and each guide says *how*.

- [ ] In Notion, go to ••• → Export → Markdown & CSV, include subpages, and unzip it into `assets/notion-guides/`.
- [ ] Before giving it to Claude Code, delete the Supplements, Injections & Peptides, Stock Tracker and PE Enhancement pages, and remove the pill and injection lines from Morning and Night.

```
STAGE: Guides library.

Import the pages in assets/notion-guides/ as Guides. Fix Notion's
broken tables. Add a Guides page (cards with emoji + title, grouped
Study / Athletic / Personal) with a reader and editor for each guide.

Turn the Morning and Night guides into routines: each timed block
becomes a step on Home's routine checklist, with its time, duration
and a link to the related guide (e.g. "Skincare" opens the Skincare
guide). Routines reset daily and never go overdue.

Guides that change often (Athletic & Lower changes weekly) get a
fixed-rules section plus a dated weekly log.

Let Ask AI and the Telegram bot search the guides when answering.
```

**Checkpoint:**

- [ ] Morning and Night show on Home as timed step checklists, and each step opens its guide.
- [ ] Athletic & Lower has fixed rules plus a weekly log.
- [ ] Asking Telegram "What's my recovery-night skincare?" answers from your guide.

## Step 11: Google Calendar and Whoop (1–2 sessions)

Google Calendar becomes your schedule view and your phone notifications. Whoop feeds your recovery and sleep into the planner so it doesn't overload you on bad days.

- [ ] Claude Code walks you through Google Cloud Console: turn on the Calendar API, create OAuth credentials, and add yourself as a test user (fine for personal use).
- [ ] At developer.whoop.com, create an app to get a client ID and secret.
- [ ] Put all the new keys in `.env.local` and in Vercel's environment settings.

```
STAGE: Google Calendar + Whoop. Do Calendar first, stop, then Whoop.

GOOGLE CALENDAR (two-way, Google OAuth):
- Create a dedicated "Life OS" calendar in my Google account.
- Push fixed events and any task I give a time to into it, with
  Google reminders (30 min before; "leave by" alerts when a travel
  time is saved).
- Read my other calendars so Upcoming, Ask AI and "Do I have time
  Thursday?" know when I'm busy.
- Changes I make in Google Calendar sync back to the app.
- Never edit or delete events outside the Life OS calendar.

WHOOP (read-only, official WHOOP developer API, OAuth):
- Pull daily recovery %, sleep (hours + performance), strain and
  workouts into my own tables with row level security.
- Small Recovery card on Home.
- Ask AI uses it: on low-recovery days suggest lighter training and
  protect sleep. Whoop workouts can count toward weekly targets
  (ask me which activity types count).

All tokens server-side only. Add both to the security checklist.
```

**Checkpoint:**

- [ ] A task you give a time to appears in Google Calendar on your phone, with a reminder.
- [ ] Moving it in Google Calendar updates the app.
- [ ] The Recovery card shows today's Whoop numbers.
- [ ] On a low-recovery day, "What should I do tonight?" goes easier on training.

## Step 12: Study pipelines and GoodNotes (1 session)

A pipeline is your learning method for a subject, as stages you can edit any time. The dashboard then shows the next action for each topic, not "study Further Maths".

Example, Further Maths: **Bicen Maths video → notes in GoodNotes → textbook questions → exam-style questions → past paper questions → review mistakes.** You're still researching the best method, so change the stages whenever you like.

- [ ] Get your exam board's specification topic list for each subject (paste it into capture or give Claude Code the PDF).
- [ ] GoodNotes has no connection for apps to read notes, so link to them instead: in GoodNotes settings turn on **Auto-Backup to Google Drive** as PDF, then paste the notebook's Drive link onto each topic.

```
STAGE: Study pipelines.

Let me define a pipeline per subject: an ordered list of stages
(e.g. Further Maths: Bicen video → GoodNotes notes → textbook
questions → exam questions → past papers → review mistakes).
Pipelines stay editable; changing stages must not lose progress.

Each subject has topics (I'll paste the spec topic list). Track the
stage each topic is at. Home and Ask AI show the NEXT action, e.g.
"FM: Complex numbers — exam questions (stage 4)", max one or two
per subject.
Each topic holds resources: video links, question sets, Google
Drive links to my GoodNotes PDFs.
Logged mistakes ("Q5 wrong — sign error") go to a review list that
comes back a few days later.
Capture understands "finished the Bicen video on matrices" and
moves that topic forward. Works from Telegram too.
```

**Checkpoint:**

- [ ] You can create the Further Maths pipeline and reorder its stages.
- [ ] Typing "finished Bicen video on matrices" moves Matrices on one stage.
- [ ] Home shows one or two next actions per subject, not a wall.
- [ ] A topic's GoodNotes link opens your notes.

## Step 13: Load your life in and start using it

**First-time brain dump (15 min).** Type these into Quick Capture one at a time and confirm each:

- Every homework and deadline you currently know about.
- Weekly targets: football sessions, gym sessions, SMC sessions.
- Context facts: your subjects, SMC date, CS tutor day, how you like to plan (flexible, not minute-by-minute).
- Resources: key SMC guides, CS videos, your GoodNotes notebook share links (one per subject).
- Then delete the sample tasks it seeded.

**Daily loop (5 minutes total):**

| When | What you do |
|---|---|
| Morning | Glance at Today. Tick routine items. |
| During the day | Capture anything the second it comes up, on your phone. |
| After school | Ask "What should I do tonight?" Do the work. Tick things off. |
| Evening | Tick the night routine. Capture anything left in your head. |
| Sunday | Ask "Plan my week." Adjust. Export a backup. |

Keep Notion untouched for the first week. If you go 7 days using only the dashboard, you can retire Notion.

## Which Claude to use, and keeping costs down

Use the cheaper model by default and switch up only for hard problems. In Claude Code, switch with `/model`.

| Job | Use |
|---|---|
| Building in Claude Code (most of the time) | Sonnet 5.5 |
| Planning the schema, stubborn bugs, final design review | Opus 5.5 |
| Helper chats in claude.ai ("this error came up") | Sonnet 5.5, in a fresh chat |
| Designing the look | Claude Design |
| Your app's AI capture (sorting what you type) | Haiku 4.5: fast and cheap |
| Your app's Ask AI and Telegram answers | Sonnet 5.5 |

When you reach Step 6, tell Claude Code: *"Use Claude Haiku 4.5 for capture classification and Claude Sonnet 5.5 for Ask AI."*

**Saving usage:**

- Chats and Claude Code share one usage limit, which resets every 5 hours.
- Do one stage per Claude Code session, and type `/clear` before the next one.

- Never paste the course into chats. It's already in `assets/`.
- Keep your Anthropic key only in `.env.local`, never exported in the terminal, or Claude Code bills your API account instead of your plan.
- Running the finished app costs API usage, not plan usage, capped by your spend limit.

## Later upgrades and fixing bugs

**Add these only after a week of real use**, one at a time, based on what annoys you:

- **Google Calendar sync** and Whoop: now built in Step 11.
- **Evening journal**: his Day 6 flow, if you want Ask AI to spot patterns over time.
- **Football session generator**: Ask AI builds a 30–40 min session from your focus areas (juggling, first touch, scanning, weak foot).
- **Automatic travel times**: a maps API so "leave in X min" works without you entering travel times.

**When something breaks**, give Claude Code this (from his Day 7):

```
Bug:
What I did:
What I expected:
What happened:
Does it happen every time?
Fix the cause, not the symptom. Don't change anything else.
```

Fix anything that loses data, duplicates tasks or shows the wrong numbers before changing how things look. And change one thing at a time, so you always know what broke it.